



# Phase 3 — Project Design Phase

System Architecture, Database Schema & Problem-Solution Fit

---

Design documentation for an AI-powered debt relief and  
financial recovery platform

Prepared as part of the project documentation set  
Document type: System Design & Data Architecture Overview

## Phase 3: Project Design Phase

This document details the system architecture, database schema, and problem-solution fit for the platform — connecting each identified borrower pain point to a concrete feature, then laying out how those features are structured technically and persisted in the database.

### 1. Problem-Solution Fit Matrix

Each core feature is designed to directly relieve a specific pain point felt by a borrower in financial distress.

| Customer Pain                         | Pain Reliever / Feature       | Value Created                                                                       |
|---------------------------------------|-------------------------------|-------------------------------------------------------------------------------------|
| High anxiety over debt payments       | DTI & EMI Ratios on Dashboard | Gives the borrower a clear, objective understanding of their financial load.        |
| Unaware of what settlement to ask for | Settlement Predictor Engine   | Suggests target percentage reductions based on delinquency and stress levels.       |
| Unable to draft professional letters  | AI Hardship Letter Generator  | Drafts print-ready formal letters automatically using Google Gemini AI.             |
| Lacks legal negotiation strategy      | AI Strategy Advisor           | Provides 3-4 bulleted negotiation tips specific to the creditor type and situation. |
| Loses track of past proposals         | Negotiation History Accordion | Persistently saves and logs all generated strategies and letters.                   |

## 2. Solution Architecture Flow

The platform uses a decoupled client-server architecture. Requests flow from the user through a presentation layer and an API router, which fans out to two specialized services — a financial processing engine and a generative-AI layer — before converging on a single persistence layer.

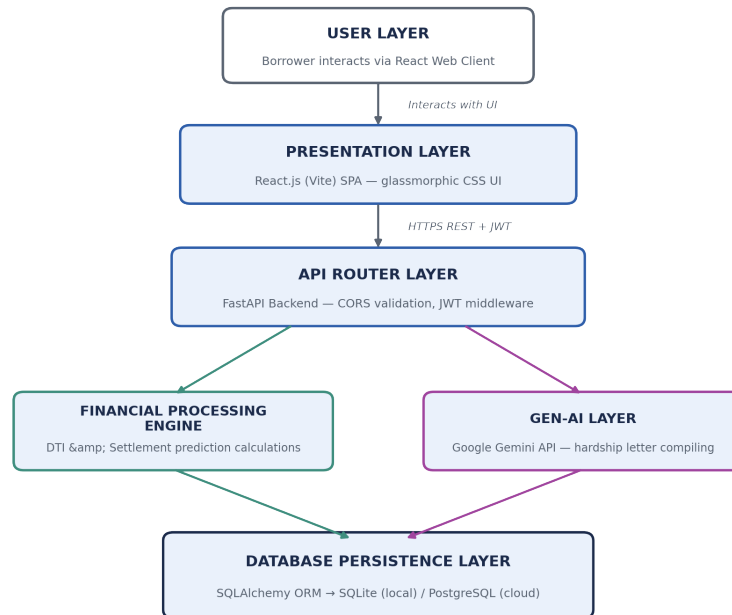


Figure 1. Decoupled client-server architecture, from UI to persistence.

**Why the fan-out matters:** keeping the Financial Processing Engine and the Gen-AI Layer as separate services (rather than one monolithic route) means the fast, deterministic DTI/settlement math never waits on a slower external Gemini API call, and either one can be scaled, cached, or retried independently.

### 3. Database Schema Reference

The database consists of six tables linked by primary and foreign keys, spanning the user's profile, their loans, calculated ratios, settlement predictions, and AI-generated output.

**Database Entity-Relationship Overview**

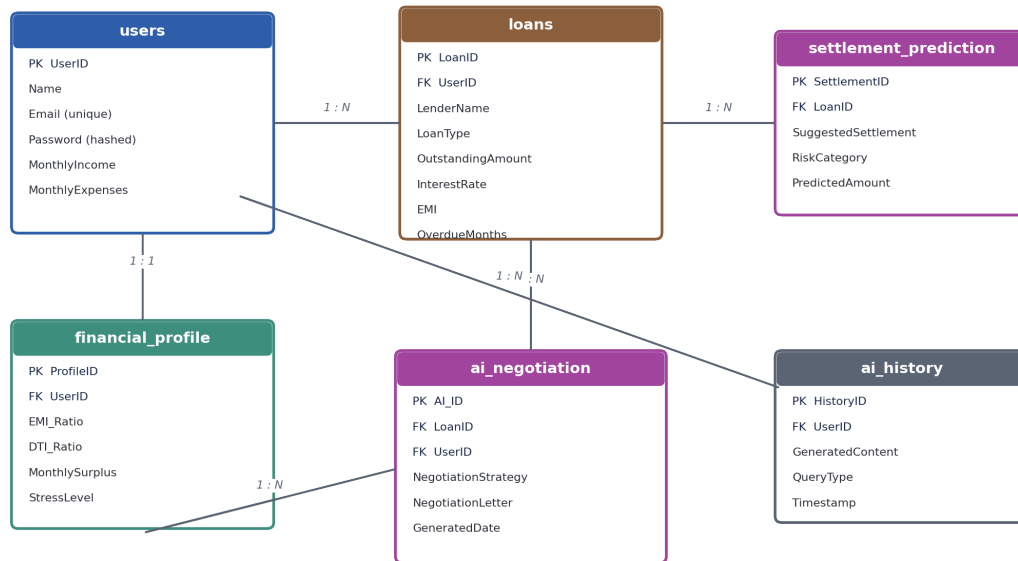


Figure 2. Entity-relationship overview of the six core tables.

### 3.1 users — Primary Profile

| users           |                                                        |
|-----------------|--------------------------------------------------------|
| PK UserID       | Integer — unique identifier for the user.              |
| Name            | String — user's full name.                             |
| Email           | String, unique — login username.                       |
| Password        | String — hashed password (never stored in plain text). |
| MonthlyIncome   | Float — user's gross monthly income.                   |
| MonthlyExpenses | Float — user's basic living expenses.                  |

### 3.2 financial\_profile — Ratios Table (1:1 with users)

| financial_profile |                                           |
|-------------------|-------------------------------------------|
| PK ProfileID      | Integer — primary key.                    |
| FK UserID         | Integer → users.UserID — one-to-one link. |
| EMI_Ratio         | Float — Total EMI / Income.               |
| DTI_Ratio         | Float — Total Debt / Annual Income.       |
| MonthlySurplus    | Float — Income – Expenses – EMIs.         |
| StressLevel       | String — Low, Medium, High, or Severe.    |

### 3.3 loans — Liability Table (1:N with users)

| loans             |                                                     |
|-------------------|-----------------------------------------------------|
| PK LoanID         | Integer — primary key.                              |
| FK UserID         | Integer → users.UserID — one-to-many link.          |
| LenderName        | String — name of the creditor.                      |
| LoanType          | String — Credit Card, Personal Loan, Mortgage, etc. |
| OutstandingAmount | Float — total balance unpaid.                       |
| InterestRate      | Float — annual percentage rate (APR).               |
| EMI               | Float — active monthly payment.                     |
| OverdueMonths     | Integer — months delinquent.                        |

### 3.4 settlement\_prediction — Calculations Table (1:N with loans)

| settlement_prediction |                                                                     |
|-----------------------|---------------------------------------------------------------------|
| PK SettlementID       | Integer — primary key.                                              |
| FK LoanID             | Integer → loans.LoanID — one-to-many link.                          |
| SuggestedSettlement   | Float — recommended payoff percentage (e.g. 45.0).                  |
| RiskCategory          | String — default litigation risk level (Low, Medium, High, Severe). |
| PredictedAmount       | Float — calculated payoff value in dollars.                         |

### 3.5 ai\_negotiation — AI Output Log

| ai_negotiation      |                                               |
|---------------------|-----------------------------------------------|
| PK AI_ID            | Integer — primary key.                        |
| FK LoanID           | Integer → loans.LoanID.                       |
| FK UserID           | Integer → users.UserID.                       |
| NegotiationStrategy | Text — bullets of advice generated by Gemini. |
| NegotiationLetter   | Text — complete hardship letter text.         |
| GeneratedDate       | DateTime — date of creation.                  |

### 3.6 ai\_history — Audit Log

| ai_history       |                                                                     |
|------------------|---------------------------------------------------------------------|
| PK HistoryID     | Integer — primary key.                                              |
| FK UserID        | Integer → users.UserID.                                             |
| GeneratedContent | Text — concise record of the event.                                 |
| QueryType        | String — “Settlement Prediction Review” or “Hardship Letter Draft”. |
| Timestamp        | DateTime — audit date.                                              |

## 4. Additional Notes & Recommendations

The following points are not part of the original design text, but are common considerations worth tracking as the schema and architecture move from design into implementation.

### 4.1 Schema & Data Integrity Notes

- Add a unique index on users.Email to enforce one account per address at the database level, not just in application code.
- Add a foreign-key index on loans.UserID and settlement\_prediction.LoanID — both are queried constantly and benefit most from indexing.
- Consider a CHECK constraint (or application-level validation) on StressLevel and RiskCategory so only the defined enum values are ever written.
- Store GeneratedDate and Timestamp in UTC consistently to avoid timezone drift between the API server and database.

### 4.2 Security & Compliance Considerations

- Hash passwords with a strong, salted algorithm (e.g. bcrypt or argon2) — never store or log plain-text passwords.
- Treat NegotiationLetter and GeneratedContent as sensitive financial PII; encrypt at rest and restrict read access by role.
- Apply row-level access control so a user's API token can only ever query rows where UserID matches their own account.
- Review regional consumer-debt and financial-advice regulations before persisting or displaying settlement percentages as guidance.

### 4.3 Scaling & Performance Notes

- Introduce a migration tool (Alembic) from the start so schema changes are versioned before the move from SQLite to PostgreSQL.
- Cache Settlement Predictor results per loan snapshot — the calculation is deterministic, so repeat requests don't need to recompute.
- Batch or queue Gemini API calls if usage grows, to smooth out latency spikes and control cost.
- Paginate the Negotiation History Accordion query rather than loading a user's full ai\_history in one call.

### 4.4 Suggested Next Steps

- Phase 4: Define the OpenAPI contract for each router (auth, financial profile, loans, predictor, AI negotiation).
- Phase 5: Prompt-engineering pass on the Gemini negotiation prompt, with test cases per hardship and creditor type.
- Phase 6: Load-test the Financial Processing Engine and Settlement Predictor under concurrent multi-loan requests.
- Ongoing: Track prediction accuracy against real-world settlement outcomes to recalibrate RiskCategory thresholds.